L4 · 数据与性能（1 亿条 / 预聚合 / 缓存）

项	内容
对应次课	S09 – S10
时长	90 分钟（完整级需课前预生成）
目标	亲手跑出"173 秒 → 0.23 毫秒"，理解索引与预聚合的差别
产出	对比数据表 + 结论（≥ 150 字）

---

1. 三档规模（按机器条件选一档）

档	命令	生成耗时	磁盘
演示级	`python tools/gen_bulk_data.py --rows 200000 --db data/bulk.db --force`	2 s	10 MB
教学级	`--rows 5000000`	30 s	260 MB
完整级	`--rows 100000000`	17 min	5 GB

> 结论在三个规模上都成立（只是数量级不同）；课堂用教学级即可。

---

2. 实验一：明细聚合（慢）

```
# ■■■■■■■■■■■■ Python ■■■■■■■■■■■
python - <<'PY'
import sys, time
sys.path.insert(0, "impl/services/agent-runtime")
sys.path.insert(0, "impl/packages/tiangong_core")
from app.analytics import AnalyticsStore
s = AnalyticsStore("data/bulk.db")
t = time.perf_counter(); r = s.summary(7); dt = (time.perf_counter()-t)*1000
print(f"source={r['source']}  {dt:.1f} ms  orders={r['orders']['total']:,}")
PY
```

记录：source = _____，耗时 = _____ ms

---

3. 实验二：建预聚合（快）

```
python tools/build_rollup.py --db data/bulk.db --rebuild
# ■■■■■■■■■■
```

记录：source = _____，耗时 = _____ ms

> 思考：为什么第二次不用改任何查询代码，就变快了？（提示：`AnalyticsStore` 自动检测 rollup）

---

4. 实验三：缓存预热与命中

```
python tools/redis_compat.py &            # ■ Redis ■■■■
python tools/warmup_cache.py --bulk-db data/bulk.db --days 1,7,30 --timelines 50

# ■■■■■■■■■■
TIANGONG_BULK_DB=data/bulk.db TIANGONG_REDIS_URL=redis://127.0.0.1:6379/0 \
  python -m granian --interface asgi app.server:app --port 8200
curl -s localhost:8200/tg-api/v1/observability/metrics
```

记录：`cache.hit_rate` = _____，`l1_hits` = _____，`l2_hits` = _____

5. 数据表（必填）

方案	数据量	耗时	备注
明细聚合（无索引优化）			
明细聚合（有 ts 索引）			
预聚合			
预聚合 + L2 命中			
预聚合 + L1 命中			

6. 结论（≥ 150 字，必须回答三点）

- 为什么加了索引反而可能更慢？
- 预聚合的代价是什么（存储/时效/维护）？
- 如果数据涨到 10 亿条，你会先改哪一层？

7. 评分（100 分）

项	分值
三组实验数据真实完整	40
结论能解释机理（不是罗列数字）	35
缓存实验与命中率记录	15
思考题	10

8. 思考题

- 预聚合表要不要每天重建？重建期间查询怎么办？
- 缓存里为什么不能放"红线判定结果"？
- 如果客户要求"实时"（秒级新鲜度），预聚合还适用吗？怎么改？